EXPECTED

Data Structures

&

Algorithms

Question Types for the Term End Examination

Drafted By: AM

Reviewed By: YM

Disclaimer

The following document is based on previous trends (2023–2025). There is a non-zero possibility that these trends will not repeat this year. Discretion of the highest order is advised. By continuing to read this document, the "reader" acknowledges that the author shall not be held liable for any academic, mental, emotional, physical, or psychological harm. The author claims no "insider information" and makes it unequivocally clear that this document is, in the most elementary sense, based on "guesswork". By reading further, the reader exonerates the author and the reviewer from all potential liabilities and damages.

"Homo in Luna ambulavit."
"Man has walked on the Moon."

Time: 3 hours Maximum Marks: 100

Answer Q1 and any four questions from Q2–Q7. Each question carries 20 marks. For revision, attempt every question and the alternative in Q6(a).

Write algorithms as numbered steps: Step 0: Start; Step 1, Step 2, ...; End. Show intermediate states. Use zero-based array indices and state any additional assumptions.

Q1. Answer briefly.

(a) An array stack has capacity 5 and is initially empty. Trace:
`Push(10), Push(20), Push(30), Peek(), Pop(),`
`Push(40), Push(50), Push(60), Push(70).`
Show the stack after every operation, returned values, and any overflow. [5]

(b) For $[4, 3, 1, 5, 2]$, show every pass of selection sort and insertion sort. State which standard implementation is stable. [5]

(c) Define the degree, depth and height of a tree node. Explain the difference between preorder, inorder and postorder traversal. State your height convention. [5]

(d) Choose and justify a suitable structure for (i) expected constant-time passenger lookup by ticket ID and (ii) repeatedly scheduling the job with the shortest running time. [5]

Q2.

(a) Explain O, Ω and Θ notation. Determine the tight time bound and auxiliary space of the following function, with reasons. Assume constant-time arithmetic. [10]

```
Analyze(A, n):
    s = 0
    for i = 0 to n-1:
        s = s + A[i]
    for i = 1 to n:
        for j = 1 to i:
            s = s + 1
    return s
```

(b) (i) Write a stack algorithm to check balanced brackets; trace it on `(a-c)*(b+c)`. [5]
(ii) Evaluate `9 3 4 * 8 + 4 / -`, showing the stack after each token and the order in which operands are popped. [5]

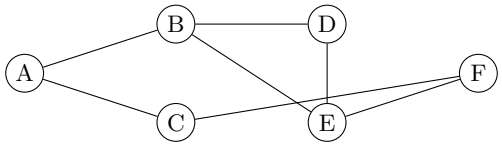
Q3.

(a) Starting with $10 \to 20 \to 30 \to 40 \to$ NULL, insert 25 after node 20, then delete the node containing 30. Draw both pointer changes. Write algorithms for insertion after a known node and deletion by value, including empty-list, head-node and not-found cases. [10]

(b) Construct a BST by inserting $50, 30, 70, 20, 40, 60, 80, 65$ in order. Draw the tree after each insertion. Explain the three deletion cases; delete 50 using its inorder successor and show every change. Give the final inorder traversal. [10]

Q4.

(a) Write BFS and DFS algorithms. Traverse the following undirected graph from A, considering neighbours alphabetically. Show the BFS queue and DFS active stack after each discovery, and give both orders. [5+5]



(b) Write merge-sort and merge algorithms. Sort $39, 9, 81, 45, 90, 27, 72, 18$, showing every split and merge. Explain the time and auxiliary-space complexities. [10]

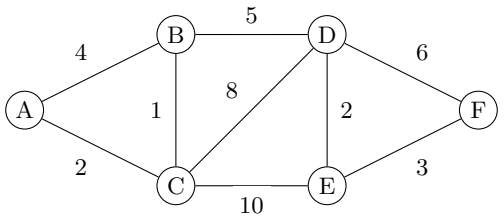
Q5.

(a) (i) Write an infix-to-postfix algorithm. Convert $((A+B)*C-D)/(E+F)$, showing the operator stack and output after each token. [7]
(ii) Convert `P Q R * + S -` to fully parenthesised infix using a stack. [3]

(b) Write the postorder traversal algorithm. Reconstruct the binary tree from the sequences below, showing every partition and node attachment. Give its preorder traversal. [10]
Inorder: $4, 8, 2, 5, 1, 6, 3, 7$.
Postorder: $8, 4, 5, 2, 6, 7, 3, 1$.

Q6.

(a) For the weighted, undirected graph below, define a minimum spanning tree. Construct one using Prim's algorithm from A, then verify it using Kruskal's algorithm. Show each chosen edge, any rejected edge inspected, and the total cost. [5+5]



OR

Write Dijkstra's algorithm. Using the same graph with A as source, show every settling and relaxation step, the final distance to each vertex, and the shortest routes to E and F. Break distance ties alphabetically. [10]

(b) Explain linear probing. Insert $27, 37, 47, 57, 15, 25, 35$ into a table of size 10 using $h(k) = k \bmod 10$. Show every probe, including wraparound, and the final table. Find the load factor and explain two disadvantages of linear probing. [10]

Q7.

(a) Write circular-queue enqueue and dequeue algorithms, including full and empty tests. Use capacity 5 and initially $front = rear = -1$. Enqueue 10,20,30,40; dequeue twice; then enqueue 50,60,70,80. Show the array, front, rear and logical FIFO order after every operation; identify any overflow. Give two applications. [10]

(b) Write Huffman's algorithm. For frequencies $A : 5$, $B : 9$, $C : 12$, $D : 13$, $E : 16$, $F : 45$, show the priority queue after every merge and draw every newly formed subtree. Assign 0 to left edges and 1 to right edges. Give all codes and the total encoded bit count; compare it with fixed-length coding, excluding codebook overhead. [10]